5주차 풀이 읽기자료

해설 사용 순서

먼저 학생 실습에서 문제를 독립적으로 풉니다. 이 자료는 힌트와 완전 해설을 포함합니다. 필요한 단계까지만 읽고 다시 풀이합니다.

표시된 예시는 강의에서 만든 예시입니다. 원문과 전체 제약은 각 공식 링크에서 확인합니다.

학생 실습

교사용 진행안

5주차 코드 내려받기

215 · 단계별 힌트

LeetCode 215: Kth Largest Element in an Array · Medium

H1. 지금까지 가장 큰 k개만 남기면 무엇이 답인가요?

H2. 새 값이 그중 최소보다 작으면 보관할 이유가 있나요?

H3. 처음 k개를 heapify하고 이후 값이 root보다 크면 heapreplace합니다.

215 · 정당성과 비용

힙은 지금까지 본 값 중 큰 k개입니다. root 이하의 새 값은 상위 k개를 바꾸지 못하고, 큰 새 값은 가장 작은 보관 값을 대체해야 합니다. 마지막 root가 k번째입니다.

n=길이. O(n log(k+1)) 시간, O(k) 보조 공간. 초기 heapify O(k). 전체 음수 힙 대안은 O(n+k log n) 시간·O(n) 공간.

215 · 반례와 검증

최소 힙에서는 각 부모의 값이 자식 이하이고, root에 전체 최솟값이 놓입니다. 중복을 set으로 없애면 예시의 답이 달라집니다.

`nums=[8,3,8,2,5], k=2 → 8`

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

215 · 구현

```
def kth_largest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)
    for index in range(k, len(nums)):
        value = nums[index]
        if value > heap[0]:
            heapq.heapreplace(heap, value)
    return heap[0]
```

56 · 단계별 힌트

LeetCode 56: Merge Intervals · Medium

H1. 시작점 순서로 보면 어느 결과 구간만 다시 검사하면 되나요?

H2. 새 end가 더 작을 때 현재 end를 줄여도 되나요?

H3. 시작점으로 정렬하고 마지막 결과 구간과 겹치면 끝을 max로 늘립니다.

56 · 정당성과 비용

처리한 접두부의 합집합은 result와 같습니다. 시작점이 증가하므로 새 구간은 마지막 결과와만 겹칠 수 있습니다. 분리되면 이후에도 앞 결과와 겹치지 않습니다.

n=구간 수. 정렬 O(n log n), 스캔 O(n); 정렬 보조 O(n), 출력 O(n). 입력을 복사 정렬합니다.

56 · 반례와 검증

[[1,8],[2,3]]에서 end=3으로 덮으면 [3,8]을 잃습니다. 반열린 시간 구간은 닿음 규칙이 다를 수 있습니다.

`intervals=[[5,7],[1,3],[3,6]] → [[1,7]]`

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

56 · 구현

```
def merge_intervals(intervals):
    result = []
    for start, end in sorted(intervals, key=lambda item: item[0]):
        if result and start <= result[-1][1]:
            result[-1][1] = max(result[-1][1], end)
        else:
            result.append([start, end])
    return result
```

200 · 단계별 힌트

LeetCode 200: Number of Islands · Medium

H1. 방문한 육지를 다시 시작점으로 세면 어떻게 되나요?

H2. 대각선도 간선인가요?

H3. 미방문 육지에서 count를 늘리고 연결된 육지를 스택으로 모두 표시합니다.

200 · 정당성과 비용

한 번 시작한 탐색은 연결 경로를 따라 그 성분의 모든 노드와 그 노드만 방문합니다. 이미 표시한 성분은 다시 시작하지 않으므로 시작 횟수가 성분 수입니다.

R,C=행·열 수. O(RC) 시간·O(RC) 보조 공간. 일반 인접 리스트에서는 O(V+E) 시간입니다.

200 · 반례와 검증

값 비교는 ==입니다. is는 객체 동일성입니다. [[“1”,”0”],[“0”,”1”]]은 대각선을 연결하지 않아 두 성분입니다.

```
grid=[["1","0","1"],["1","0","0"]] → 2
```

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

200 · 구현

```
def islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen, count = set(), 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1" or (row, col) in seen:
                continue
            count += 1
            stack = [(row, col)]
            seen.add((row, col))
            while stack:
                r, c = stack.pop()
                for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                    if (0 <= nr < rows and 0 <= nc < cols
                            and grid[nr][nc] == "1" and (nr,nc) not in seen):
                        seen.add((nr,nc))
                        stack.append((nr,nc))
    return count
```

994 · 단계별 힌트

LeetCode 994: Rotting Oranges · Medium

H1. 처음부터 여러 시작점이 있으면 하나씩 기다려야 하나요?

H2. 같은 셀을 두 번 큐에 넣지 않으려면 언제 바꾸나요?

H3. 모든 시작점을 같은 큐에 넣고 레벨별로 처리합니다. 새 셀은 enqueue할 때 즉시 2로 바꿉니다.

994 · 정당성과 비용

큐의 레벨은 어떤 시작점에서든 그 셀까지의 최소 간선 수입니다. 모든 간선의 전파 시간이 같으므로 먼저 발견한 시간이 최소입니다. 큐가 비어도 fresh가 남으면 경로가 없습니다.

R,C=행·열 수. O(RC) 시간, 복사 board와 큐 O(RC) 보조 공간.

994 · 반례와 검증

[[2,0,1]]은 -1, [[0,2]]는 0입니다. 다음 레벨까지 같은 분에 연쇄 처리하면 시간을 작게 계산합니다.

`grid=[[2,1,1],[0,1,0]] → 2`

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

994 · 구현

```
def rot_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    board = [row[:] for row in grid]
    queue, fresh, minutes = deque(), 0, 0
    for r in range(rows):
        for c in range(cols):
            if board[r][c] == 2:
                queue.append((r,c))
            fresh += board[r][c] == 1
    while queue and fresh:
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] == 1:
                    board[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr,nc))
        minutes += 1
    return minutes if fresh == 0 else -1
```

실행과 참고자료

```
cd code
python demo_09.py
python demo_10.py
python verify.py
```

표준 라이브러리 import는 `solutions.py`에 있습니다. LeetCode에서는 공식 메서드 서명에 맞춰 함수를 연결합니다.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

중간 점검의 답과 근거

1. 8과 2. 중복 8을 두 번 셉니다.
2. `[[1,8]]`과 `[[1,6]]`. 포함 구간의 끝으로 기존 끝을 줄이지 않으며, 닫힌 구간은 끝점이 닿으면 합칩니다.
3. 2. 대각선은 연결하지 않습니다.
4. -1과 0. 첫 지도는 신선한 셀에 경로가 없고, 둘째는 처음부터 신선한 셀이 없습니다.

개념 질문 해설

정렬된 리스트의 삽입은 위치를 이진 탐색해도 원소 이동 때문에 O(n)입니다. 최소 힙은 전체 정렬이 아니며 루트만 전역 최솟값입니다.

BFS의 첫 발견이 최소 비용이 되는 조건은 모든 간선 비용이 같다는 것입니다. 비용이 다르면 간선 수가 적은 경로가 더 비쌀 수 있습니다.